CAREERMATE AI | KNOWLEDGE BASE | VOLUME 1 OF 3

Resume & Portfolio Knowledge Base

Resume writing, ATS optimization, GitHub and LinkedIn presence, and how to showcase AI/ML projects for internships and entry-level roles

Intended audience

Students, freshers and career changers preparing for internships and entry-level roles: Python Developer, Data Analyst, ML Engineer, AI Engineer.

Scope note

This volume contains general, widely applicable guidance. It does not describe the policies or hiring requirements of any specific organization. Always follow an employer's own instructions when they differ from this guide.

Version 1.0 | resume_portfolio_guide.pdf

Table of Contents

About This Guide . . . 6

Purpose and scope . . . 6

General advice versus organization-specific requirements . . . 6

How to read the examples . . . 6

Terminology used in this guide . . . 7

1. Resume Fundamentals and Structure . . . 8

1.1 What a resume is and what it must do . . . 8

1.2 The four questions every resume answers . . . 8

1.3 Recommended length . . . 8

1.4 Resume formats . . . 8

1.5 Recommended section order for freshers . . . 9

1.6 The master resume and tailored versions . . . 9

1.7 Personal information conventions . . . 9

1.8 Section purpose checklist . . . 9

2. Professional Resume Formatting . . . 11

2.1 Principles of clean formatting . . . 11

2.2 Layout and typography . . . 11

2.3 Header format . . . 11

2.4 Bullet points . . . 11

2.5 Dates and consistency . . . 12

2.6 File format and file name . . . 12

2.7 What to avoid in formatting . . . 12

2.8 Formatting checklist . . . 12

3. Writing Effective Professional Summaries . . . 13

3.1 What a professional summary is . . . 13

3.2 A simple formula . . . 13

3.3 Examples for each role . . . 13

3.4 Before and after . . . 14

3.5 Summary for a fresher with no experience . . . 14

3.6 Common mistakes in summaries . . . 14

3.7 Summary checklist . . . 14

4. Technical Skills and Soft Skills Presentation . . . 15

4.1 Why the skills section matters . . . 15
4.2 Group skills by category . . . 15
4.3 Skill sets by target role . . . 15
4.4 Be honest about proficiency . . . 15
4.5 Match keywords without keyword stuffing . . . 15
4.6 Showing soft skills . . . 16
4.7 Skills section checklist . . . 16

5. Writing Strong Project Descriptions . . . 17

5.1 Why projects are central for freshers . . . 17
5.2 Anatomy of a project entry . . . 17
5.3 The action-task-result pattern . . . 17
5.4 Metrics: use real numbers, and know what to do without them . . . 17
5.5 Action verbs by purpose . . . 18
5.6 Handling tutorial and course projects . . . 18
5.7 Team projects . . . 18
5.8 Sample project entries . . . 18
5.9 Project description checklist . . . 18

6. Work Experience and Internship Experience Sections . . . 20

6.1 Purpose of the experience section . . . 20
6.2 Structure of an experience entry . . . 20
6.3 Writing internship bullets . . . 20
6.4 Experience that is not a formal job . . . 20
6.5 Transferable experience from non-technical jobs . . . 21
6.6 Employment gaps . . . 21
6.7 Experience section checklist . . . 21

7. Education and Certifications . . . 22

7.1 Education section . . . 22
7.2 Relevant coursework . . . 22
7.3 Certifications: how to choose . . . 22
7.4 Achievements, hackathons and activities . . . 23
7.5 Education and certification checklist . . . 23

8. ATS (Applicant Tracking System) Optimization . . . 24

8.1 What an ATS is and what it does . . . 24
8.2 How a resume is processed . . . 24

8.3 Myths and facts . . . 24
8.4 Formatting rules for ATS-friendly resumes . . . 24
8.5 Keyword alignment: a practical method . . . 25
8.6 Testing your resume for parsing . . . 25
8.7 ATS optimization checklist . . . 25

9. Common Resume Mistakes . . . 26

9.1 Why mistakes matter . . . 26
9.2 Table of common mistakes and fixes . . . 26
9.3 Technical resume specific mistakes . . . 26
9.4 Self-review exercise . . . 27
9.5 Mistakes checklist . . . 27

10. Resume Evaluation Criteria . . . 28

10.1 How a resume is evaluated . . . 28
10.2 Evaluation rubric . . . 28
10.3 Interpreting your score . . . 28
10.4 What evaluators look for at each role level . . . 29
10.5 Red flags evaluators notice . . . 29
10.6 Worked evaluation example . . . 29
10.7 Final evaluation checklist . . . 29

11. GitHub Portfolio Best Practices . . . 30

11.1 Why GitHub matters for technical applicants . . . 30
11.2 Profile setup . . . 30
11.3 Repository structure . . . 30
11.4 README template . . . 30
11.5 Commit and branch habits . . . 31
11.6 Notebook hygiene . . . 31
11.7 Contribution graph: a realistic view . . . 31
11.8 Quality over quantity . . . 31
11.9 GitHub portfolio checklist . . . 31

12. LinkedIn Profile Optimization . . . 32

12.1 Role of LinkedIn in applications . . . 32
12.2 Profile elements and how to write them . . . 32
12.3 Headline examples . . . 32
12.4 Sample About section . . . 32

12.5 Visibility and activity . . . 33
12.6 Consistency with the resume . . . 33
12.7 LinkedIn checklist . . . 33

13. How to Showcase AI/ML Projects . . . 34

13.1 What reviewers want to see . . . 34
13.2 Project ladder: from tutorial to original . . . 34
13.3 Project workflow to document . . . 34
13.4 Metrics and honesty . . . 34
13.5 Showcasing generative AI and RAG projects . . . 35
13.6 Sample ML project README summary . . . 35
13.7 Sample RAG project README summary . . . 35
13.8 Matching projects to roles . . . 36
13.9 Showcase checklist . . . 36

14. Resume Examples for Python Developer, Data Analyst, ML Engineer and AI Engineer . . 37

14.1 How to use these examples . . . 37
14.2 Sample 1: Python Developer (fresher) . . . 37
14.3 Sample 2: Data Analyst (fresher) . . . 38
14.4 Sample 3: ML Engineer (entry-level) . . . 39
14.5 Sample 4: AI Engineer (entry-level) . . . 40
14.6 What the samples have in common . . . 40

15. Resume Improvement Strategies . . . 41

15.1 An iterative improvement process . . . 41
15.2 Rewrite patterns . . . 41
15.3 Closing gaps with real work . . . 41
15.4 30-day improvement plan . . . 42
15.5 Final pre-submission checklist . . . 42
15.6 Frequently asked questions . . . 42

About This Guide

Purpose and scope

This knowledge base explains how to design, write, evaluate and improve a resume and an online professional presence (GitHub and LinkedIn) for technical roles. It is written for students, freshers and career changers who are applying for internships and entry-level positions as a Python Developer, Data Analyst, Machine Learning (ML) Engineer or AI Engineer.

Each chapter is self-contained. A reader, or a retrieval system, can open any chapter or section on its own and find the definitions, explanations, examples and checklists needed to act on that topic without reading the rest of the document.

General advice versus organization-specific requirements

Everything in this guide is **general advice** based on widely accepted resume and portfolio practice. Hiring is not standardized. Different employers, countries, recruiters and applicant tracking systems (ATS) behave differently.

- **General advice** answers questions such as "How long should a fresher resume be?" or "What makes a project description strong?". It is a sensible default, not a rule.
- **Organization-specific requirements** are instructions given by a particular employer, such as a required file format, a mandatory form field, a word limit, or a request to include or omit a photograph. These always override the general advice in this guide.
- This guide does not describe the hiring policies, scoring methods or legal obligations of any named company. When a section says "many employers" it describes a common pattern, not a guarantee.

> **Note** If a job posting gives explicit application instructions, follow them exactly, even if they differ from this guide.

How to read the examples

All names, companies, projects and numbers in sample resumes and sample sections are **fictional** and exist only to demonstrate structure and wording. Do not copy a sample verbatim. Replace every detail with true information about your own work. Never invent results, tools, employers or credentials.

Terminology used in this guide

Term	Meaning in this guide
Resume	A short, targeted document (usually one page for freshers) that summarizes your skills, projects, experience and education for a specific role. Many people in India and elsewhere also call this a CV.
CV (Curriculum Vitae)	In some regions and in academia, a longer and more detailed record. In this guide, "resume" is used for the short job-application document.
ATS	Applicant Tracking System: software employers use to receive, store, parse, search and track job applications.
Parsing	The ATS step that extracts text and fields (name, email, skills, dates) from an uploaded resume file.
Keyword	A skill, tool or phrase from a job description (for example "SQL" or "scikit-learn") that a recruiter or ATS may search for.
Fresher	A candidate with little or no full-time professional experience, such as a final-year student or recent graduate.
Bullet point	A single short line in an experience or project section that states what you did and the outcome.
Impact	The measurable or observable result of your work: time saved, accuracy reached, users served, errors reduced.
Portfolio	A collection of your work, usually GitHub repositories, a personal site and demos, that proves your skills.
Baseline	A simple first model or method used as a reference point so improvements can be measured honestly.

1. Resume Fundamentals and Structure

1.1 What a resume is and what it must do

A resume is a marketing document, not an autobiography. Its single job is to earn a conversation: an interview, a screening call or a reply from a recruiter. It does this by showing, quickly and credibly, that you can do the work described in a specific job posting.

Recruiters and hiring managers often review many applications for one opening, so a resume is usually read quickly at first. A reader decides whether to look more closely based on a few signals: a clear headline or summary, relevant skills, recognizable projects or experience, and clean formatting. Everything on the page should help that decision.

> **Definition** **Resume** - a one- to two-page, role-targeted summary of your skills, projects, experience and education, written to convince a recruiter to contact you.

1.2 The four questions every resume answers

A good technical resume answers four questions within the first half page:

1. **Who are you professionally?** (name, target role, location, contact links)
2. **What can you do?** (skills and tools relevant to the role)
3. **What have you built or achieved?** (projects, internships, results)
4. **What is your background?** (education, certifications)

If a reader cannot answer all four quickly, the resume needs restructuring, not more content.

1.3 Recommended length

Candidate type	Typical length	Reason
Student or fresher with projects only	1 page	Limited experience; brevity forces prioritization
Fresher with internships and several strong projects	1 page (2 only if every line is relevant)	Readers prefer concise, relevant content
Professional with several years of experience	1-2 pages	More history to summarize
Academic or research CV	Longer, as the field expects	Publications and grants need space

For internships and entry-level roles, one page is the safe default. Cutting a weak bullet is better than shrinking the font to fit it.

1.4 Resume formats

Three layouts are common. Choose one deliberately.

- **Reverse-chronological:** Lists experience and education from most recent to oldest. Most employers and ATS expect this format. It works well when you have internships or relevant projects with dates.
- **Functional (skills-based):** Groups content by skill instead of by job. It can hide a lack of experience and many recruiters distrust it. It is generally not recommended for technical roles.

- **Hybrid (combination):** Starts with a skills or projects section, then lists experience and education in reverse-chronological order. This is often the best choice for freshers because projects come first and are still dated.

> **Tip** For most students and freshers applying to AI/ML, data and Python roles, a hybrid layout with the order Summary, Skills, Projects, Experience, Education works well.

1.5 Recommended section order for freshers

Order	Section	Purpose
1	Header (name and contact)	Lets the recruiter reach you and see your work
2	Professional summary (2-3 lines)	States target role, core strengths and focus
3	Technical skills	Gives keyword coverage and a quick skill scan
4	Projects	Main evidence of ability for freshers
5	Experience or internships	Shown above projects if you have a relevant internship
6	Education	Degree, institution, year, relevant coursework
7	Certifications	Only those relevant to the role
8	Achievements or activities (optional)	Hackathons, open-source, competitions

If you have a relevant internship, move Experience above Projects. If your degree is your strongest credential and you have little else, Education may go higher, but projects should still appear early for technical roles.

1.6 The master resume and tailored versions

Keep a **master resume**: a long private document listing every project, skill, certification and achievement with full detail. For each application, create a **tailored version** by selecting only the items that match the job description and reordering them so the most relevant content comes first.

Tailoring does not mean inventing skills. It means choosing which true facts to emphasize. A Data Analyst version of the same candidate might lead with SQL dashboards and Excel analysis, while the ML Engineer version leads with model training projects.

1.7 Personal information conventions

- Include: full name, phone number, professional email address, city and country (the street address is unnecessary), LinkedIn URL and GitHub URL. A portfolio link is optional.
- A photograph, date of birth, marital status, gender, religion and parents' names are generally not expected on technical resumes and are best left out unless an employer explicitly asks. Conventions differ by country, so follow the posting's instructions.
- Use a simple, professional email address based on your name.

1.8 Section purpose checklist

- [ ] The resume targets one role family, not "any job".
- [ ] The first third of the page shows the target role and relevant skills.
- [ ] Every section earns its space by supporting the target role.

- [ ] Dates, titles and links are consistent throughout.
- [ ] The document fits one page without tiny fonts.

2. Professional Resume Formatting

2.1 Principles of clean formatting

Formatting should make the resume easy to scan and safe for parsing software. Plain and consistent beats decorative. Use one column, standard section headings, readable fonts and generous but controlled spacing.

2.2 Layout and typography

Element	Recommended practice
Font	A standard, readable font such as Calibri, Arial, Helvetica, Georgia or Times New Roman
Body size	About 10-11 pt (never below 10 pt)
Name	Larger than body text, about 16-20 pt
Section headings	Bold, slightly larger, consistent style, such as capitals or a thin rule
Margins	About 0.5-1 inch (1.3-2.5 cm) on all sides
Line spacing	Single or slightly more; keep space between sections
Colour	Black or dark grey text; one subtle accent colour at most
Columns	A single column is safest; multi-column layouts can parse in the wrong order

2.3 Header format

The header should let a recruiter contact you and open your work in one glance. Put the most important links on one or two lines.

> **Aarav Mehta**
> Python & ML Developer | Pune, India
> aarav.mehta@example.com | +91 90000 00000 | linkedin.com/in/aarav-mehta-example | github.com/aarav-example

Note Write links as readable text and make them clickable hyperlinks. A printed copy or a parser that strips hyperlinks should still show the address.

2.4 Bullet points

- Start each bullet with a strong action verb in the correct tense: past tense for finished work, present tense for current work.
- Keep each bullet to one or two lines.
- Use 2-4 bullets per project or role. Fewer, stronger bullets beat long lists.
- Put the most important bullet first.
- Avoid first-person pronouns ("I", "my"). The resume is implicitly about you.

2.5 Dates and consistency

Use one date format throughout, such as "Jun 2025 - Aug 2025" or "06/2025 - 08/2025". Align dates to the right margin consistently. Mark current work with "Present". Check that capitalization, punctuation (end each bullet with a full stop or none, but be consistent), and tool names (for example "PostgreSQL", "scikit-learn", "LangChain") are identical everywhere.

2.6 File format and file name

- **PDF** keeps layout stable on every device and is the usual choice, unless the employer requests DOCX. Export a text-based PDF, not a scanned image or a screenshot.
- **DOCX** is requested by some employers or recruiters who want to edit the file.
- Use a clear file name such as `Aarav_Mehta_ML_Engineer_Resume.pdf` rather than `resume_final_v7_new.pdf`.

2.7 What to avoid in formatting

- Graphics, icons, skill-rating bars, charts, photos and text boxes that parsers may ignore or scramble.
- Headers and footers that hold critical information (some parsers skip them).
- Tables used for layout, unusual fonts, and more than two font styles.
- Dense paragraphs. Use short bullets instead.

2.8 Formatting checklist

- [ ] One column, standard headings (Summary, Skills, Projects, Experience, Education).
- [ ] Font 10 pt or larger, consistent spacing.
- [ ] Contact details and links correct and clickable.
- [ ] Dates and tool names consistent.
- [ ] Exported as a text-based PDF with a professional file name.
- [ ] Opened on another device to confirm nothing shifted.
- [ ] Copy-pasted into a plain text editor to confirm the reading order is correct.

3. Writing Effective Professional Summaries

3.1 What a professional summary is

A professional summary is a two- to three-line statement at the top of the resume that tells the reader who you are, what you do and what you offer. It replaces the older "career objective", which described what the candidate wanted rather than what the employer would gain.

Definition **Professional summary** - a short opening statement that names your target role, your strongest relevant skills and your most credible proof.

3.2 A simple formula

Use this pattern: **[Role or level] + [core skills] + [proof] + [focus or goal related to the employer's need].**

- Role or level: "Final-year BCA student", "Entry-level data analyst".
- Core skills: two to four skills that match the posting.
- Proof: a project, internship or result, stated honestly.
- Focus: the type of work you want to do for the employer.

3.3 Examples for each role

Python Developer (fresher): Computer science student with hands-on experience building REST APIs and automation scripts in Python, Flask and SQLite. Built a task-management API with authentication and unit tests. Seeking a Python developer internship to contribute to backend features and write clean, tested code.

Data Analyst (fresher): Analytics-focused graduate skilled in SQL, Excel and Power BI. Analyzed a public retail dataset of 50,000 rows to identify seasonal sales patterns and built an interactive dashboard. Looking for a data analyst internship to turn raw data into clear business insights.

ML Engineer (fresher): Machine learning enthusiast with practical experience in scikit-learn and PyTorch, from data cleaning to model evaluation. Built an image classifier using transfer learning and compared it against a baseline model. Seeking an ML engineering internship to work on model training and evaluation pipelines.

AI Engineer (fresher): Developer building LLM applications with Python, LangChain and LangGraph. Created a retrieval-augmented generation (RAG) assistant over PDF documents using ChromaDB, with a basic evaluation set to measure answer quality. Seeking an AI engineering internship to build reliable generative AI features.

Note The numbers and projects above are fictional examples. Use only facts you can defend in an interview.

3.4 Before and after

Weak summary	Why it is weak	Stronger version
Hardworking student seeking a challenging role in a reputed company to utilize my skills.	Generic, no skills, no proof, could belong to anyone	Computer science student skilled in Python and SQL, with two data analysis projects on public datasets, seeking a data analyst internship.
Passionate about AI and want to learn new things.	States feelings, not capability	Student developer who built a sentiment classifier in scikit-learn and a small chatbot using an LLM API, seeking an AI internship.
Expert in Python, ML, DL, NLP, SQL, AWS, and more.	Unsupported claim of expertise	Python developer with project experience in ML (scikit-learn) and SQL, familiar with cloud basics.

3.5 Summary for a fresher with no experience

Lead with education and projects. Describe what you have built and what you are learning, and name the role you target. Avoid apologizing for a lack of experience: simply state your strengths.

3.6 Common mistakes in summaries

- Using vague adjectives ("hard-working", "team player", "passionate") with no proof.
- Writing more than four lines.
- Listing every tool you have ever touched.
- Claiming "expert" or "proficient" in skills you have only practiced briefly.
- Using the same summary for every job.

3.7 Summary checklist

[ ] Two to three lines at most.
[ ] Names the target role or role family.
[ ] Includes two to four relevant skills from the job description.
[ ] Includes one piece of real proof.
[ ] Contains no filler adjectives and no first-person pronouns.

4. Technical Skills and Soft Skills Presentation

4.1 Why the skills section matters

The skills section provides a fast scan for recruiters and a keyword match for ATS software. It must be accurate, grouped and aligned to the job. Every skill listed is something an interviewer may ask about, so list only skills you can discuss and use.

4.2 Group skills by category

Grouping makes the section easy to scan. Put the most role-relevant group first.

Programming: Python (primary), SQL, basic Java
Libraries and frameworks: pandas, NumPy, scikit-learn, PyTorch, Flask
Data and databases: PostgreSQL, SQLite, ChromaDB, Excel
Tools: Git, GitHub, Jupyter, VS Code, Docker (basics)
Concepts: Data cleaning, feature engineering, model evaluation, REST APIs

4.3 Skill sets by target role

Role	Skills to emphasize
Python Developer	Python, OOP, Flask or Django or FastAPI, REST APIs, SQL, Git, unit testing, Linux basics
Data Analyst	SQL, Excel, Python (pandas, Matplotlib), Power BI or Tableau, statistics basics, data cleaning, dashboarding
ML Engineer	Python, NumPy, pandas, scikit-learn, PyTorch or TensorFlow, model evaluation, feature engineering, MLOps basics, Git
AI Engineer	Python, LLM APIs, prompt design, LangChain, LangGraph, RAG, vector databases, evaluation, APIs, Docker basics

4.4 Be honest about proficiency

Avoid percentage bars, star ratings and "expert" labels. They are subjective and cannot be verified. Instead, show level through context.

- **Strong claim backed by evidence:** "Python - used in 5 projects, including a deployed API."
- **Honest limited claim:** "Docker (basics)" or "Familiar with AWS S3 and EC2".
- **Do not list** tools you cannot discuss. Skills you only watched a tutorial about belong in a learning plan, not on the resume.

4.5 Match keywords without keyword stuffing

Read the job description and note the tools it names. If you truly have a skill, use the same wording the posting uses ("scikit-learn", not "sklearn only"; "PostgreSQL" or "Postgres"). Writing the full name with the acronym once helps both humans and ATS, for example "Natural Language Processing (NLP)".

Warning Do not hide invisible keywords in white text or paste long lists of unrelated tools. Recruiters and some systems detect this, and it damages trust.

4.6 Showing soft skills

Soft skills such as communication, teamwork, problem solving and leadership cannot be proven by listing them. Show them through evidence inside project, internship and activity bullets.

Soft skill	Weak (list only)	Strong (evidence)
Communication	Good communication skills	Presented project findings to a 40-person class and wrote the README used by two teammates
Teamwork	Team player	Collaborated with 3 teammates using Git branches and pull requests to deliver a hackathon prototype in 24 hours
Problem solving	Strong problem solver	Diagnosed a 20% accuracy drop caused by data leakage and corrected the train-test split
Leadership	Natural leader	Led a 4-member project team, assigning tasks and running weekly check-ins
Learning ability	Quick learner	Learned PyTorch in 3 weeks and applied it to a working image classifier

Note The numbers above illustrate the writing pattern and are fictional. Use real numbers or leave numbers out.

4.7 Skills section checklist

- [ ] Skills grouped into clear categories, most relevant group first.
- [ ] No rating bars, percentages or "expert" claims without evidence.
- [ ] Every listed skill can be demonstrated in a project or discussed in an interview.
- [ ] Keywords from the job description included where honest.
- [ ] Soft skills shown through project and activity bullets, not listed.

5. Writing Strong Project Descriptions

5.1 Why projects are central for freshers

When you lack job experience, projects are your proof of ability. A project description should tell the reader what problem you solved, what you built, how you built it and what happened as a result.

5.2 Anatomy of a project entry

A strong entry has a title line and two to four bullets.

- **Title line:** Project name, a short descriptor, key technologies, a link to the code or demo, and optionally the date.
- **Bullet 1 (what and why):** The problem and the solution in one line.
- **Bullet 2 (how):** Techniques, tools and technical decisions.
- **Bullet 3 (result):** Measurable outcome or observable result.
- **Bullet 4 (optional):** Deployment, testing, documentation or teamwork.

5.3 The action-task-result pattern

Each bullet should combine an **action verb**, the **task or method** and the **result or purpose**. A widely used variation is: "Accomplished [X], as measured by [Y], by doing [Z]."

Weak bullet	Strong bullet
Worked on a machine learning project.	Built a customer-churn classifier in scikit-learn using logistic regression and random forest, reaching 0.84 ROC-AUC on a held-out test set.
Made a chatbot using LangChain.	Developed a PDF question-answering assistant using LangChain, ChromaDB and an LLM API, returning answers with source page references.
Did data analysis on sales data.	Analyzed 3 years of sales records with SQL and pandas to identify the 5 highest-margin product categories; built a Power BI dashboard used for the final presentation.
Responsible for creating a web app.	Designed a Flask REST API with JWT authentication and 12 endpoints, covered by 20 pytest unit tests.

5.4 Metrics: use real numbers, and know what to do without them

Numbers make claims concrete, but only if they are true and explained.

- **Prefer relevant metrics:** accuracy, F1-score, ROC-AUC, RMSE, latency, number of records, time saved, number of users or test cases.
- **Always give context** for a model metric: on which dataset and which split (test set, cross-validation).
- **Compare against a baseline** whenever possible: "improved F1 from 0.71 (baseline) to 0.82".
- **If you have no metric,** describe scale or scope instead: "processed 10 PDF reports", "supports 3 user roles", "handles missing values and outliers in 14 columns".
- Never fabricate a number. Interviewers often ask how a figure was computed.

5.5 Action verbs by purpose

Purpose	Verbs
Building	Built, developed, implemented, designed, engineered, created
Improving	Improved, optimized, reduced, increased, refactored, streamlined
Analyzing	Analyzed, evaluated, compared, investigated, visualized, quantified
Deploying	Deployed, containerized, automated, integrated, published
Collaborating	Collaborated, led, coordinated, documented, presented, mentored

5.6 Handling tutorial and course projects

Many learners build the same Titanic or MNIST project. Such a project can still appear on a resume if you add something of your own: a new dataset, a different model comparison, error analysis, a deployed demo or a clear written explanation. If a project follows a tutorial closely, describe your extensions rather than presenting it as original design, and be ready to explain every line.

5.7 Team projects

State your own contribution precisely: "Built the data preprocessing pipeline and model evaluation module in a 4-person team." Do not take credit for teammates' work.

5.8 Sample project entries

Customer Churn Predictor | Python, pandas, scikit-learn, Streamlit | github.com/aarav-example/churn-predictor

- Cleaned and encoded a telecom dataset of 7,000 customers, handling missing values and class imbalance with stratified sampling.
- Compared logistic regression, random forest and gradient boosting using 5-fold cross-validation; selected random forest with 0.84 ROC-AUC on the test set.
- Deployed an interactive Streamlit app where users enter customer details and see churn probability with the top contributing features.

Document Q&A Assistant (RAG) | Python, LangChain, ChromaDB, LLM API | github.com/aarav-example/doc-qa

- Built a retrieval-augmented generation pipeline that splits PDFs into chunks, stores embeddings in ChromaDB and answers questions with cited source pages.
- Created a 30-question evaluation set to compare chunk sizes; chose 800 characters with 100 overlap based on the best retrieval hit rate.
- Added a fallback response when retrieved context is not relevant, reducing unsupported answers in manual testing.

Note Project names, links and numbers above are fictional demonstrations of the pattern.

5.9 Project description checklist

- [ ] Each project has a title, technologies and a working link.
- [ ] Bullets start with strong, varied action verbs.

- [ ] At least one bullet per project states a result, scale or outcome.
- [ ] Metrics are truthful and include context such as dataset and split.
- [ ] Your own role is clear in team projects.
- [ ] The strongest, most role-relevant projects appear first, and weak ones are removed.

6. Work Experience and Internship Experience Sections

6.1 Purpose of the experience section

The experience section proves that you have worked in a real setting: with deadlines, teammates, code reviews, data that is messy and users who depend on the result. For freshers, an internship is valuable even when short. If you have no formal internship, you can still present relevant practical experience under a suitable heading.

6.2 Structure of an experience entry

Each entry has a header and two to four bullets.

- **Header line:** Role title, organization, location (city or "Remote") and dates.
- **Optional context line:** One line describing the team or product if it is not obvious.
- **Bullets:** Action, method, result. Lead with the strongest.

Data Analyst Intern | Example Retail Analytics Pvt. Ltd. (fictional) | Remote | Jun 2025 - Aug 2025

- Wrote SQL queries to extract and reconcile weekly sales data from 4 tables, replacing a manual spreadsheet process.
- Built a Power BI dashboard tracking returns by product category, reviewed weekly by the category manager.
- Documented data definitions for 18 metrics, reducing repeated questions from the sales team.

6.3 Writing internship bullets

Interns usually contribute to a part of a larger system. Describe your part clearly and honestly.

1. State the task and the tool: "Wrote Python scripts to clean customer feedback data."
2. State the purpose or outcome: "...so the support team could categorize complaints faster."
3. Add scale or evidence if you have it: "...covering 2,000 weekly records."
4. Mention collaboration when relevant: "...reviewed in weekly code reviews with the team lead."

Warning Do not disclose confidential information such as customer names, private data, internal code or proprietary figures. Describe the work in general terms unless you have permission to share specifics.

6.4 Experience that is not a formal job

Practical experience can be listed under headings such as Experience, Internships, Open Source Contributions or Freelance Work, provided the description is accurate.

Type	How to present it
Unpaid or virtual internship	List as an internship with the organization name, dates and tasks actually performed
Freelance project	Use "Freelance Developer" with the client type (for example "small business") and the deliverable
Open-source contribution	Name the repository, describe the fix or feature, and link the pull request

Type	How to present it
Hackathon	Name the event, your role, what was built in the time limit, and any recognition
Research assistant or college project with faculty	State the topic, your task, and the tools used
Volunteer technical work	Describe the system built or the data handled and who used it

6.5 Transferable experience from non-technical jobs

If you have worked in a non-technical job, include it briefly when it shows reliability, responsibility, communication or domain knowledge. Write one or two bullets that connect it to the target role, for example "Reconciled daily transaction records with 100% accuracy checks, building attention to detail relevant to data quality work." Keep it short and place it after technical content.

6.6 Employment gaps

A short gap rarely needs an explanation on the resume. If a longer gap is likely to be asked about, one honest line is enough, such as "Full-time self-study in Python and machine learning with 3 projects completed". Be ready to discuss the gap calmly in an interview.

6.7 Experience section checklist

- [ ] Each role has title, organization, location or "Remote", and dates.
- [ ] Bullets describe your own actions and outcomes, not the team's general duties.
- [ ] No confidential data is shared.
- [ ] Informal experience is labeled accurately and not presented as a paid full-time job.
- [ ] Verbs are in the correct tense and varied.

7. Education and Certifications

7.1 Education section

Education confirms your qualification and sometimes acts as an eligibility filter. Keep it concise.

- **Include:** degree and specialization, institution, location, expected or actual graduation year.
- **Optional:** CGPA or percentage (include it when it is strong or when the employer asks), relevant coursework, honors and scholarships.
- **Order:** Reverse-chronological. Fresh graduates can place Education above Experience only when it is their strongest credential.
- **School-level results** (grade 10 and 12) are generally omitted for technical roles unless an application form requires them.

Bachelor of Computer Applications (BCA) | Example University, India | Expected 2028
- Relevant coursework: Data Structures, Database Management Systems, Python Programming, Statistics
- CGPA: 8.1/10 (fictional)

Note Degrees earned through open or distance learning are valid qualifications. List the degree and university name accurately, and let your projects demonstrate your practical skills.

7.2 Relevant coursework

List four to six courses that match the target role. For a Data Analyst: Statistics, Database Management, Data Visualization. For an ML Engineer: Linear Algebra, Probability, Machine Learning. Do not list every subject.

7.3 Certifications: how to choose

Certifications show initiative, but they carry different weight. Employers generally value demonstrated skill more than certificates alone.

Certification type	Value for a resume
Recognized vendor or professional certification (for example cloud fundamentals or database certifications)	Often relevant when it matches the job's stack; verify the exam is current
Reputable university or platform courses with graded assignments	Useful, especially when you built a project from the course
Short badges or participation certificates	Low weight; list only if directly relevant and short on space
Unrelated certificates	Remove them

- List the certification name, issuing organization and year. Add a verification link when one exists.
- Mark unfinished programs honestly: "In progress, expected Dec 2026".
- Prefer a few relevant certifications over a long list.
- Pair a certification with a project that applies it so the skill is demonstrated, not only claimed.

Certifications
- Machine Learning Specialization, Example Learning Platform (2025) - verification link

- SQL for Data Analysis, Example University Online (2025)

7.4 Achievements, hackathons and activities

An optional section can list competitions, scholarships, coding contest ranks and leadership in technical clubs. Include only achievements you can describe with context: "Top 10 of 120 teams at a college hackathon" is clearer than "Hackathon winner". Never exaggerate a ranking.

7.5 Education and certification checklist

- [ ] Degree, institution, location and year are accurate.
- [ ] Coursework is limited to relevant subjects.
- [ ] Certifications are relevant, recent and verifiable where possible.
- [ ] In-progress items are labeled as such.
- [ ] No unrelated or padded entries.

8. ATS (Applicant Tracking System) Optimization

8.1 What an ATS is and what it does

An applicant tracking system is software that employers use to collect applications, store candidate information, let recruiters search and filter candidates, and move people through hiring stages. Large employers and many recruiting agencies use one, and the systems differ in features and configuration.

> **Definition** **ATS** - Applicant Tracking System, a database and workflow tool that receives resumes, extracts their text into fields, and lets recruiters search, sort and track applications.

8.2 How a resume is processed

1. You upload a file through a job portal.
2. The system **parses** the file: it extracts text and tries to identify your name, contact details, job titles, dates, education and skills.
3. The extracted information is stored in a candidate profile.
4. A recruiter searches or filters profiles using keywords, locations, dates or screening questions, and reads the matching resumes.

How much automatic ranking or rejection happens depends on how the employer has configured the system. Some employers rely mostly on recruiter review, others use knockout questions such as work authorization or required degree. The goal of ATS optimization is simple: make sure your information is **read correctly** and **found** when a recruiter searches.

8.3 Myths and facts

Common belief	A more accurate view
An ATS reads your resume and automatically rejects most candidates	Behavior varies by employer. Many systems primarily organize applications for human review; filters depend on configuration
You need a secret keyword trick	Using the job description's honest terminology is enough. Hidden or repeated keywords can backfire
Fancy designs impress the system	Complex layouts, columns, graphics and text boxes can confuse parsers
A single universal ATS format exists	No universal format exists, but plain, standard structure works broadly

8.4 Formatting rules for ATS-friendly resumes

- Use a single-column layout with standard headings: Summary, Skills, Projects, Experience, Education, Certifications.
- Use standard fonts and simple bullet characters.
- Avoid tables for layout, text boxes, images, icons, charts and skill bars.
- Keep contact details in the main body, not in a page header or footer.

- Use a text-based PDF or DOCX. Avoid scanned images.
- Write dates in a consistent, common format.
- Spell out acronyms once with the full name, for example "Natural Language Processing (NLP)".

8.5 Keyword alignment: a practical method

1. Copy the job description and highlight required skills, tools and responsibilities.
2. Group them into must-have, nice-to-have and soft skills.
3. Compare them with your master resume and list every one you honestly have.
4. Use the posting's exact wording in the skills section and in relevant bullets.
5. Place the most important keywords where a human reader will also see them: the summary, skills, and the first bullets of your projects.

Job description excerpt: "Experience with Python, pandas, SQL, and building dashboards. Familiarity with Git. Strong communication."

Resume alignment:

- Skills: Python (pandas, NumPy), SQL (PostgreSQL), Power BI, Git and GitHub
- Project bullet: Built a Power BI dashboard from a PostgreSQL database, presented findings to a 30-student class

Warning Never add skills you do not have just to match a posting. A keyword match that collapses in the interview wastes everyone's time.

8.6 Testing your resume for parsing

- Copy all text from the PDF and paste it into a plain text editor. The order should read naturally: header, summary, skills, and so on.
- Check that your name, email and phone number appear correctly at the top.
- Ensure no characters turn into symbols and that bullets are not scrambled.
- Upload to a portal that shows a parsed preview if one is available and compare the fields.

8.7 ATS optimization checklist

- [ ] Standard section headings and a single-column layout.
- [ ] No images, icons, text boxes or skill bars.
- [ ] Keywords from the job description used honestly in skills and bullets.
- [ ] Contact details in the main body of the page.
- [ ] Plain-text copy test passed.
- [ ] File named clearly and saved as text-based PDF (or DOCX if requested).

9. Common Resume Mistakes

9.1 Why mistakes matter

A reader with many applications to review often looks for reasons to move on. Small errors signal carelessness, and large problems hide your real strengths. The mistakes below are the most frequent in fresher resumes and each has a direct fix.

9.2 Table of common mistakes and fixes

Mistake	Why it hurts	Fix
Typos and grammar errors	Suggests lack of care, especially for roles that require precision	Proofread slowly, use a spell checker, read aloud, and ask another person to check
Generic summary or objective	Does not show relevance	Write a role-specific summary with skills and proof
Task lists instead of results	Hides what you actually achieved	Use action-task-result bullets with outcomes
Listing every tool ever used	Creates doubt about depth and invites hard questions	Keep skills you can demonstrate; group them
Skill rating bars and percentages	Subjective and unverifiable; poor for parsing	Show level through projects and honest wording
Broken or missing project links	Reader cannot verify your work	Test every link; make repositories public
Copy-pasted tutorial projects	Looks identical to hundreds of others	Add your own dataset, analysis, deployment or comparison
Exaggeration or false claims	Collapses in interviews and can damage your reputation	Claim only what you can explain and demonstrate
Two-page resume full of filler	Dilutes strong content	Cut to one page and keep the best material
Unprofessional email address	Reduces credibility	Use a name-based email
Inconsistent formatting and tenses	Looks careless	Apply one style and review every section
Irrelevant personal details	Takes space and may be unnecessary	Remove unless the employer requests it
Missing keywords from the job description	Weak match for search and screening	Tailor skills and bullets honestly
Large paragraphs	Hard to scan	Break into short bullets

9.3 Technical resume specific mistakes

- **Listing "AI/ML" without evidence:** Naming a field is not proof. Show a project with data, model, evaluation and outcome.
- **Metrics without context:** "95% accuracy" is meaningless without dataset, split and baseline. A model with 95% accuracy on a dataset where 95% of examples belong to one class is not impressive.

- **Only notebooks, no engineering:** For many roles, show clean code, a README, and ideally a demo or API in addition to notebooks.
- **Overclaiming deep learning:** If you used a pre-trained model through an API, say "used", not "built from scratch".
- **No version control evidence:** A GitHub link with consistent, readable work strengthens the resume.
- **Mentioning courses instead of outputs:** "Completed course X" is weaker than "Built Y after course X".

9.4 Self-review exercise

Read the resume aloud and ask, line by line:

1. Does this line help me get the target role?
2. Is it true, and can I explain it for five minutes?
3. Does it show a result or only an activity?
4. Would a stranger understand it without extra context?

Delete or rewrite any line that fails two of these questions.

9.5 Mistakes checklist

- [ ] Zero spelling or grammar errors after two separate reads.
- [ ] No unverifiable claims, rating bars or inflated titles.
- [ ] All links tested in a private browser window.
- [ ] No irrelevant or confidential information.
- [ ] Every project has an explicit result or scope.

10. Resume Evaluation Criteria

10.1 How a resume is evaluated

Evaluators, whether recruiters, hiring managers or mentors, judge a resume on **relevance, evidence, clarity and credibility**. Understanding these criteria lets you score your own resume honestly before you send it.

10.2 Evaluation rubric

Score each criterion from 0 (absent) to 5 (excellent). A total below the mid-range signals specific areas to improve.

Criterion	What excellent (4-5) looks like	What weak (0-2) looks like
Role relevance	Content clearly targets the role; skills and projects match the posting	Generic content, no connection to the role
Evidence of skill	Projects with tools, methods, results and links	Skills listed without proof
Impact and results	Bullets show outcomes, metrics with context or clear scope	Task lists with no outcomes
Clarity and structure	Standard sections, easy scan, one page, consistent style	Dense text, confusing order, inconsistent style
Technical depth	Choices explained (why this model, why this database), evaluation described	Tool names only, no reasoning
Credibility	Claims plausible and verifiable; links work	Inflated claims, broken links, unclear role in team work
ATS and format	Single column, standard headings, parseable text	Columns, graphics, scanned file
Language quality	Strong verbs, no errors, concise	Typos, vague words, long sentences
Professional presence	Working GitHub and LinkedIn that match the resume	Missing or inconsistent profiles

10.3 Interpreting your score

Total (out of 45)	Interpretation	Action
36-45	Strong	Fine-tune wording and tailor to each posting
26-35	Solid but uneven	Fix the two lowest criteria first
16-25	Needs significant work	Rewrite summary, projects and skills; remove filler
0-15	Early draft	Rebuild from the structure in chapters 1 to 5

> **Note** This rubric is a self-assessment tool, not the scoring system of any employer. Real evaluators weigh criteria differently depending on the role.

10.4 What evaluators look for at each role level

- **Internship screening:** Learning ability, fundamentals, projects that show initiative, and clean communication.
- **Entry-level screening:** Evidence of applying skills to realistic problems, teamwork, and readiness to contribute with guidance.
- **Technical review:** Whether you understand your own projects, so each bullet must be defensible in detail.

10.5 Red flags evaluators notice

- Dates that do not add up or overlap without explanation.
- Skills listed that never appear in any project or experience.
- Identical project descriptions to common public tutorials, with no original contribution.
- Repository links that are empty, private or contain only copied code.
- Buzzwords with no specifics: "leveraged cutting-edge AI to drive synergy".

10.6 Worked evaluation example

Weak project bullet: "Developed a deep learning model for image classification using TensorFlow."
Feedback: Which dataset? Which architecture? What metric and what result? Was it trained from scratch or fine-tuned? No link. Score: Evidence 1/5, Impact 1/5, Technical depth 1/5.

Improved bullet: "Fine-tuned a pre-trained ResNet-18 on a 5-class plant disease dataset (4,200 images) with data augmentation, reaching 91% test accuracy versus 74% for a small CNN baseline; code and results at github.com/example/plant-disease."
Feedback: Names data, model, technique, metric and baseline, with a link. Score: Evidence 4/5, Impact 4/5, Technical depth 4/5.

Note The example figures are fictional and illustrate the pattern, not real benchmarks.

10.7 Final evaluation checklist

- [ ] Resume scored on every criterion above, with the weakest two fixed.
- [ ] Peer or mentor review completed with written feedback.
- [ ] Each project explainable in a two-minute verbal summary.
- [ ] Resume and online profiles tell the same story.

11. GitHub Portfolio Best Practices

11.1 Why GitHub matters for technical applicants

GitHub is often the first place a technical reviewer goes to verify your resume. A clean profile with a few well-documented repositories is stronger evidence than a long list of tools. Reviewers usually do not read all your code: they look at pinned repositories, README files, project structure and signs of careful work.

Definition **Portfolio (technical)** - a curated set of public projects, with documentation and demos, that shows what you can build.

11.2 Profile setup

- Use a professional username and a clear profile photo or avatar.
- Fill in the name, short bio (role focus, such as "Python and ML developer"), location and links.
- Create a **profile README** (a repository named exactly like your username) with a short introduction, your focus areas, featured projects and contact links. Keep it concise.
- **Pin 4-6 repositories** that best match the roles you target. Order them by relevance.
- Archive or make private old experiments that do not represent your current quality.

11.3 Repository structure

A reviewer should understand a repository in under a minute.

```
project-name/
|-- README.md            # what, why, how to run, results
|-- requirements.txt     # pinned dependencies (or pyproject.toml)
|-- .gitignore           # excludes data dumps, secrets, virtual environments
|-- LICENSE              # optional, e.g. MIT
|-- data/                # small sample data or download instructions
|-- notebooks/           # exploration notebooks, cleaned and ordered
|-- src/                 # reusable Python modules
|-- tests/               # unit tests where practical
|-- app/                 # API or UI code (Streamlit, Flask, FastAPI)
`-- docs/                # screenshots, diagrams, model notes
```

11.4 README template

A README is the most important file in any repository. Use clear headings.

Project title - one-sentence description of the problem and solution.
Demo: link or screenshot. **Tech stack:** Python, pandas, scikit-learn, Streamlit.
Problem: who needs this and why. **Data:** source, size, license, preprocessing steps.
Approach: models or methods tried, baseline, evaluation metrics.
Results: table with metrics on the test set and a short interpretation. State limitations honestly.
How to run: numbered install and run commands that work on a clean machine.
Project structure: short tree of folders. **Future work:** 2-3 realistic next steps.
Author and contact.

11.5 Commit and branch habits

- Make small, meaningful commits with clear messages: "Add data cleaning for missing ages" rather than "update".
- Use branches and pull requests for larger changes, even on solo projects, to show workflow familiarity.
- Avoid committing generated files, large datasets, model binaries and virtual environments. Use `.gitignore`, and note where to download large files.
- Tag a release when a project reaches a stable version.

> **Warning** Never commit API keys, passwords, tokens or `.env` files. If a secret is committed, revoke it immediately at the provider, because deleting it from later commits does not remove it from history. Use environment variables and provide a `.env.example` with placeholder values.

11.6 Notebook hygiene

- Run the notebook from top to bottom before committing so outputs are consistent.
- Add markdown cells that explain each step and conclusion.
- Remove debugging cells and unused imports.
- Move reusable functions into `src/` modules for larger projects.

11.7 Contribution graph: a realistic view

A busy contribution graph is not a requirement and does not prove skill by itself. Quality and clarity matter more than daily green squares. Consistent work over months is a better signal than a burst of commits copied from tutorials.

11.8 Quality over quantity

Four excellent repositories beat twenty empty ones. Include at least one project that is end-to-end: data, model or logic, evaluation, a way to run it, and documentation.

11.9 GitHub portfolio checklist

- [ ] Profile README, bio and links completed.
- [ ] 4-6 relevant repositories pinned.
- [ ] Every pinned repository has a README with run instructions, results and screenshots or a demo link.
- [ ] No secrets, personal data or large binaries in any repository.
- [ ] Dependencies listed so a stranger can reproduce the project.
- [ ] Consistent naming, readable commit messages and clean folder structure.
- [ ] Repository links on the resume work in a private browser window.

12. LinkedIn Profile Optimization

12.1 Role of LinkedIn in applications

LinkedIn works as a searchable, public version of your resume. Recruiters use it to find candidates by role, skills and location, and they often check it after seeing your resume. It also supports networking and direct outreach (covered in the internship guide).

12.2 Profile elements and how to write them

Element	Guidance
Photo	Clear, well-lit, face visible, professional background
Headline	Role plus skills or focus, not just "Student". Example: "Aspiring ML Engineer, Python, scikit-learn, PyTorch, BCA student"
About section	3-5 short paragraphs or lines: who you are, what you build, tools, what you are looking for, how to contact you
Experience	Internships, freelance and project-based work using the same bullet style as the resume
Projects	Add each key project with a description and link to the repository or demo
Education	Degree, institution, years, relevant coursework
Skills	Add 15-30 relevant skills and arrange the top three that matter most for your target role
Featured	Pin your best project, GitHub profile or a short article or demo
Custom URL	Shorten to a name-based URL and use it on your resume
Certifications	Relevant ones with credential links

12.3 Headline examples

- "Python Developer (Fresher) | Flask, REST APIs, SQL | Open to internships"
- "Data Analyst | SQL, Excel, Power BI | Turning data into decisions"
- "ML Engineer in training | scikit-learn, PyTorch, model evaluation"
- "AI Engineer (entry-level) | LLM applications, RAG, LangGraph"

12.4 Sample About section

I am a computer applications student who builds practical machine learning and generative AI projects in Python.

Recently I built a RAG assistant over PDF documents using LangChain and ChromaDB, and a churn prediction app with scikit-learn and Streamlit. I enjoy turning messy data into reliable, well-documented tools.

Tools: Python, SQL, pandas, scikit-learn, PyTorch, LangChain, LangGraph, Git.

I am looking for an AI or data internship where I can contribute to real projects and learn from experienced engineers. You can reach me by LinkedIn message or at the email below.

Note The About text is a fictional pattern. Replace tools and projects with your own true ones.

12.5 Visibility and activity

- **Keywords:** Use role and skill terms in your headline, About, experience and skills, because recruiters search with them.
- **Activity:** Occasionally share a project write-up, what you learned, or a thoughtful comment. Consistency over time matters more than volume.
- **Recommendations:** Ask a faculty member, mentor or internship supervisor for a specific recommendation, and offer to write one in return.
- **Open to work settings:** LinkedIn provides options to signal that you are open to work, either visible to everyone or only to recruiters. Choose based on your comfort.
- **Privacy:** Review which contact details are public. Share a professional email only.

12.6 Consistency with the resume

Job titles, dates, project names and skills should match your resume. Inconsistencies create doubt. The LinkedIn profile can be slightly longer than the resume, but the story must be the same.

12.7 LinkedIn checklist

- [ ] Professional photo and keyword-rich headline.
- [ ] About section with skills, projects and your goal.
- [ ] Projects and experience added with links.
- [ ] 15 or more relevant skills, top three set deliberately.
- [ ] Custom URL set and added to the resume.
- [ ] Details consistent with the resume and GitHub.

13. How to Showcase AI/ML Projects

13.1 What reviewers want to see

For AI and ML roles, reviewers look for evidence of the full workflow: framing a problem, understanding data, building a baseline, evaluating honestly, improving with reasons, and communicating the result. A project that shows judgment is more convincing than one that merely shows a high number.

13.2 Project ladder: from tutorial to original

Level	Description	Resume value
1. Tutorial replication	Follow a course project on a standard dataset	Low alone; useful for learning
2. Extended tutorial	Same method with a new dataset, added comparisons or error analysis	Moderate
3. Original problem	Your own question, data you collected or combined, clear evaluation	High
4. Deployed project	An API or app others can try, with documentation and tests	High, especially for ML and AI engineering roles
5. Team or real-user project	Used by real people or built in collaboration	Very high when your role is clear

13.3 Project workflow to document

1. **Problem statement:** What are you predicting or generating, and why does it matter?
2. **Data:** Source, size, license, class balance, cleaning steps, and any bias or limitations.
3. **Baseline:** A simple method (majority class, linear model, keyword search) to compare against.
4. **Model and experiments:** Which models, features and hyperparameters, and what you learned.
5. **Evaluation:** Metrics suited to the problem (F1 or ROC-AUC for imbalanced classification, RMSE or MAE for regression), plus error analysis of mistakes.
6. **Deployment or demo:** Streamlit, Gradio, FastAPI or a notebook walkthrough, with screenshots.
7. **Limitations and next steps:** Honest statements of what the system cannot do.

13.4 Metrics and honesty

- Report test-set results, and avoid tuning on the test set.
- Mention how you split data and whether there was leakage risk.
- For imbalanced data, accuracy alone is misleading; report precision, recall or F1.
- Show a comparison table instead of a single number.

Model	Accuracy	F1 (churn class)	ROC-AUC
Majority-class baseline	0.73	0.00	0.50
Logistic regression	0.80	0.58	0.84
Random forest	0.82	0.61	0.86

Note The results above are fictional and only illustrate a comparison layout. A majority-class baseline shows why accuracy alone can mislead on imbalanced data.

13.5 Showcasing generative AI and RAG projects

Generative AI projects are easy to start and easy to make look impressive, so reviewers look for engineering discipline.

- **Explain the architecture:** Document loading, chunking, embedding model, vector store, retrieval, prompt and generation model. A simple diagram helps.
- **Show evaluation:** Create a small question set with expected answers or source pages, and report retrieval hit rate or manual answer quality. Compare at least one design choice, such as chunk size or number of retrieved chunks.
- **Handle failure:** Describe what the system does when context is missing, such as declining to answer or asking for clarification.
- **Mention cost and latency considerations** and any guardrails against unsafe or unsupported outputs.
- **State which parts you built:** Calling an API is valid, but describe it as using a model, not training one.
- **Agent and workflow projects (for example with LangGraph):** Show the state design, nodes, conditional routing and how you tested each path.

13.6 Sample ML project README summary

Plant Disease Classifier - classifies leaf images into 5 disease categories to help farmers identify problems early.

Data: 4,200 public images, 80/10/10 train/validation/test split, augmentation applied to training data only.

Approach: compared a small CNN baseline with a fine-tuned ResNet-18 using transfer learning.

Results: test accuracy 91% versus 74% baseline (fictional). Most errors occurred between two visually similar diseases.

Limitations: images taken in controlled lighting; performance on field photos is untested.

Run: `pip install -r requirements.txt` then `streamlit run app/app.py`.

13.7 Sample RAG project README summary

Policy Document Assistant - answers questions about uploaded PDF documents with page citations.

Pipeline: PDF loader, recursive text splitter (800 characters, 100 overlap), embedding model, ChromaDB, top-4 retrieval, LLM answer with sources.

Evaluation: 30 hand-written questions; correct source page retrieved in 24 of 30 cases (fictional); answers reviewed manually for faithfulness.

Failure handling: if retrieved chunks have low similarity, the assistant replies that it could not find the answer in the documents.

Limitations: tables inside PDFs are extracted poorly; no multilingual support.

13.8 Matching projects to roles

Role	Project ideas that fit
Python Developer	REST API with authentication and tests, automation tool, web scraper with data storage
Data Analyst	SQL analysis of a business dataset, dashboard with a written insights summary, A/B test analysis
ML Engineer	Classification or regression with careful evaluation, model comparison, a deployed prediction API
AI Engineer	RAG assistant with evaluation, LangGraph workflow with routing, LLM-powered tool with guardrails

13.9 Showcase checklist

- [ ] Each project states a clear problem and the dataset used.
- [ ] A baseline and an honest evaluation are included.
- [ ] A demo, screenshots or run instructions exist.
- [ ] Limitations and next steps are written down.
- [ ] You can explain every design choice in an interview.

14. Resume Examples for Python Developer, Data Analyst, ML Engineer and AI Engineer

14.1 How to use these examples

The four samples below are for **fictional candidates** and use fictional employers and results. They show structure, emphasis and wording for entry-level applicants. Adapt the pattern to your own true background. Do not copy the text.

Role	What to lead with	Emphasis in projects
Python Developer	Programming, APIs, testing, Git	Clean code, tests, deployment, databases
Data Analyst	SQL, Excel, visualization	Business questions, insights, dashboards
ML Engineer	ML fundamentals, evaluation	Data preparation, experiments, reproducibility
AI Engineer	LLM applications, RAG, orchestration	Architecture, evaluation, reliability

14.2 Sample 1: Python Developer (fresher)

Neha Kulkarni | Python Developer | Nagpur, India
neha.k@example.com | +91 90000 00001 | linkedin.com/in/neha-k-example | github.com/neha-example

SUMMARY

Computer science graduate with hands-on experience building REST APIs and automation tools in Python, Flask and SQL. Writes tested, documented code and uses Git daily. Seeking a Python developer internship.

TECHNICAL SKILLS

Languages: Python, SQL, basic JavaScript | **Frameworks:** Flask, FastAPI (basics) | **Databases:** PostgreSQL, SQLite
Tools: Git, GitHub, Docker (basics), Postman, pytest, Linux command line

PROJECTS

Task Manager API | Flask, PostgreSQL, JWT, pytest | github.com/neha-example/task-api

- Designed 14 REST endpoints with JWT authentication and role-based access for users and admins.
- Wrote 25 pytest tests covering authentication and CRUD operations; documented the API with OpenAPI.
- Containerized the service with Docker so it runs with a single command.

Expense Report Automation | Python, pandas, openpyxl

- Automated monthly expense reports from CSV exports, replacing a manual spreadsheet routine and cutting preparation time from about 2 hours to 10 minutes (own measurement).

EXPERIENCE

Python Intern, Example Software Pvt. Ltd. (fictional), Remote | Jun 2025 - Aug 2025

- Fixed 9 backend bugs in an internal inventory service and added input validation to 3 endpoints.
- Participated in weekly code reviews and followed the team's pull request workflow.

EDUCATION

B.Sc. Computer Science, Example University, 2025 | Coursework: Data Structures, DBMS, Operating Systems

14.3 Sample 2: Data Analyst (fresher)

Rohan Iyer | Data Analyst | Chennai, India

rohan.i@example.com | +91 90000 00002 | linkedin.com/in/rohan-i-example | github.com/rohan-example

SUMMARY

Analytics-focused graduate skilled in SQL, Excel, Python and Power BI. Analyzed a 50,000-row retail dataset to identify seasonal sales trends and built an interactive dashboard. Seeking a data analyst internship.

TECHNICAL SKILLS

Analysis: SQL (joins, window functions, CTEs), Excel (pivot tables, lookups), Python (pandas, Matplotlib), descriptive statistics

Visualization: Power BI, Tableau Public | **Databases:** PostgreSQL, MySQL | **Tools:** Git, Jupyter

PROJECTS

Retail Sales Insights Dashboard | SQL, Power BI | link

- Cleaned and joined 4 tables in PostgreSQL; wrote queries using window functions to rank products by monthly margin.
- Built a 3-page Power BI dashboard (sales trend, category performance, returns) with filters for region and month.
- Summarized 5 recommendations in a one-page report presented to a class of 40.

Customer Survey Analysis | Python, pandas, Seaborn

- Cleaned 1,200 survey responses, handled missing values and compared satisfaction scores across 4 customer segments.

EXPERIENCE

Data Analytics Intern, Example Analytics (fictional), Remote | Jun 2025 - Aug 2025

- Reconciled weekly sales data across systems with SQL, reducing mismatches found during review.
- Documented 15 metric definitions used in team reports.

EDUCATION

B.Com (Computer Applications), Example University, 2025 | Certification: Data Analysis with SQL (2025)

14.4 Sample 3: ML Engineer (entry-level)

Ananya Rao | Machine Learning Engineer (entry-level) | Bengaluru, India

ananya.r@example.com | +91 90000 00003 | linkedin.com/in/ananya-r-example | github.com/ananya-example

SUMMARY

Computer science graduate with project experience in the full ML workflow: data cleaning, feature engineering, training, evaluation and deployment. Built a transfer-learning image classifier and a churn prediction API. Seeking an ML engineering internship.

TECHNICAL SKILLS

ML and DL: scikit-learn, PyTorch, feature engineering, cross-validation, hyperparameter tuning, transfer learning

Data: pandas, NumPy, SQL, Matplotlib | **Engineering:** Python, FastAPI, Docker (basics), Git, pytest

PROJECTS

Plant Disease Classifier | PyTorch, torchvision, Streamlit | link

- Fine-tuned ResNet-18 on 4,200 leaf images with augmentation, reaching 91% test accuracy versus 74% for a small CNN baseline (fictional).
- Performed error analysis with a confusion matrix and identified two visually similar classes as the main source of mistakes.

Customer Churn Prediction API | scikit-learn, FastAPI, Docker | link

- Compared logistic regression, random forest and gradient boosting using 5-fold cross-validation; selected the model by ROC-AUC (0.86 on held-out data).
- Served predictions through a FastAPI endpoint with input validation and a Dockerfile; added 12 unit tests.

EXPERIENCE

ML Intern, Example AI Labs (fictional), Remote | Jan 2026 - Mar 2026

- Prepared and labeled a 3,000-sample text dataset and trained a baseline classifier used for model comparison.

EDUCATION

B.Tech Computer Science, Example Institute, 2026 | Coursework: Linear Algebra, Probability, Machine Learning

14.5 Sample 4: AI Engineer (entry-level)

Imran Sheikh | AI Engineer (entry-level) | Hyderabad, India
imran.s@example.com | +91 90000 00004 | linkedin.com/in/imran-s-example | github.com/imran-example

SUMMARY
Developer building LLM applications with Python, LangChain and LangGraph. Created a RAG assistant with a small evaluation set and a multi-step workflow with conditional routing. Seeking an AI engineering internship.

TECHNICAL SKILLS
GenAI: LLM APIs, prompt design, RAG, embeddings, LangChain, LangGraph, ChromaDB
Engineering: Python, FastAPI, Streamlit, SQL, Docker (basics), Git | **ML basics:** scikit-learn, evaluation metrics

PROJECTS
Document Q&A Assistant (RAG) | LangChain, ChromaDB, LLM API, Streamlit | link
- Built a pipeline that chunks PDFs, stores embeddings in ChromaDB and answers with page-level citations.
- Evaluated 30 questions across 3 chunk sizes; selected 800 characters with 100 overlap, which retrieved the correct page most often (fictional results).
- Added a fallback reply when retrieval confidence is low to reduce unsupported answers.

Support Ticket Router (LangGraph) | LangGraph, Python, LLM API | link
- Designed a graph with classification, conditional routing and reply-drafting nodes; tested each route with 20 example tickets.
- Logged node inputs and outputs to debug misrouted tickets and improved the classification prompt using the failures.

EXPERIENCE
AI Developer Intern, Example Tech (fictional), Remote | Feb 2026 - Apr 2026
- Prototyped an internal FAQ assistant and wrote evaluation questions with the support team.

EDUCATION
B.Sc. Information Technology, Example University, 2026 | Certification: Generative AI fundamentals course (2025)

14.6 What the samples have in common

- A short role-specific summary and a grouped skills section.
- Projects with links, tools, methods and an outcome or scope.
- Honest labels for internships and course certifications.
- One page, standard headings and simple formatting.

15. Resume Improvement Strategies

15.1 An iterative improvement process

A resume improves through cycles of drafting, feedback and testing, not through a single rewrite.

1. **Collect:** Update the master resume with new projects, results and skills every month.
2. **Target:** For each application, pick the role family and the top six requirements from the job description.
3. **Tailor:** Reorder and reword true content to match those requirements.
4. **Review:** Run the rubric in chapter 10, a plain-text parsing test and a proofreading pass.
5. **Get feedback:** Ask a mentor, senior student or engineer to read it for five minutes and tell you what they remember.
6. **Track:** Record which version you sent and what response you received, then adjust.

15.2 Rewrite patterns

Pattern	Before	After
Add the result	Created a sentiment analysis model.	Trained a sentiment classifier on 20,000 reviews, reaching 0.88 F1 against a 0.76 baseline (fictional).
Add the method	Built a chatbot using an LLM.	Built a retrieval-based chatbot using embeddings and ChromaDB so answers cite the source document.
Add scale	Cleaned the dataset.	Cleaned a 50,000-row dataset by removing duplicates, fixing 6 inconsistent date formats and imputing missing values.
Show ownership	Worked with a team on an app.	Implemented the authentication module and 8 API endpoints in a 4-person team project.
Cut filler	Responsible for working on various tasks related to data.	Wrote SQL queries to prepare weekly reports for the sales team.

15.3 Closing gaps with real work

If the resume looks thin for a target role, improve the underlying evidence rather than the wording.

- **No relevant project:** Build one scoped project in 2-4 weeks that matches the role and document it well.
- **Projects without results:** Add an evaluation, a baseline comparison and error analysis.
- **No deployment:** Wrap a model in a simple API or a Streamlit app and add run instructions.
- **No experience:** Look for open-source issues, virtual internships, freelance mini-projects or hackathons.
- **Weak fundamentals:** Practice Python, SQL and data structures problems, since interviews test them.

15.4 30-day improvement plan

Week	Focus	Output
1	Audit	Score the resume with the rubric; list top three weaknesses; clean GitHub pinned repositories
2	Evidence	Add results, baselines and READMEs to the two strongest projects
3	Rewrite	Rewrite summary, skills and bullets; test plain-text parsing; update LinkedIn to match
4	Feedback	Collect two reviews; finalize role-specific versions; start tracking applications

15.5 Final pre-submission checklist

- [ ] The resume is tailored to the specific posting using true information.
- [ ] Contact details, links and file name are correct.
- [ ] Spelling, tenses and formatting are consistent.
- [ ] Projects and skills can each be explained for several minutes.
- [ ] The resume fits one page and passes the plain-text copy test.
- [ ] GitHub and LinkedIn match the resume.
- [ ] The employer's own instructions have been followed exactly.

15.6 Frequently asked questions

Q1. Should I include a photo on my resume?

Answer: Usually not for technical roles, unless the employer or country convention requires it. Follow the posting's instructions.

Q2. Is a one-page resume mandatory?

Answer: No, but for internships and entry-level roles one page is the safest default because it forces you to keep only relevant content.

Q3. Can I list a skill I am still learning?

Answer: Yes, if you label it honestly, for example "PyTorch (learning, 1 project)", or place it in a short learning line. Do not present it as a strong skill.

Q4. Should I include projects from courses?

Answer: Yes, when they are relevant, but add your own extensions and be ready to explain every part. Prefer original projects when you have them.

Q5. How often should I update my resume?

Answer: Review it monthly and whenever you finish a project, earn a relevant certification or complete an internship.